

AI 辅助内核开发 多智能体工作流（技术评审细节版）

面向技术方案评审：强调状态机、工件契约、失败闭环、质量闸门、预算控制与 patch-ready 条件。

Artifact-Driven

Review-Oriented

Patch-Ready

评审关注点

职责边界是否清晰

状态机

生成 / 审核能否多轮收敛

Debug 效率

人工 Gate 是否放在高风险处

探索

规划

生成

审核

调试

系统边界与非目标

适用场景

RISC-V / KVM / 跨架构差距收敛
Kconfig / defconfig / selftest /
kunit 一致性问题
小步、可回滚、可解释的改动

非目标

不追求无人监督自动发 patch
不把“编过了”当作最终成功
不允许单个 Agent 包办全流程

成功标准

问题真实
方案最小
日志完整
patch-ready

关键约束

每轮只跑 1 个 issue
主实现单写分支
审核独立运行

Design doctrine

探索 -> 规划 -> 生成 -> 审核 -> 调试 -> patch-ready

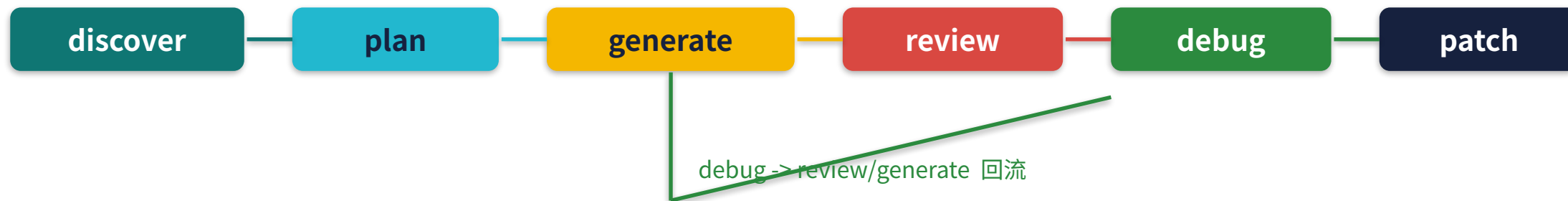
规则：

1. 全流程工件化
2. 生成 / 审核允许多轮
3. Debug 不得直接放行
4. 所有外部写操作前必须人工 Gate

为什么不是单模型串到底

内核上下文高度依赖历史与架构语义
实现者自审会放大遗漏风险
把失败拆给 Failure-Analyzer / Fix-Agent 更利于归因
并行审核比“实现者复读”更稳

控制面状态机与停止条件



workflow.yaml stop_conditions

```
max_rounds_generation_review: 3
max_rounds_debug: 2
stop if decision == PATCH_READY
stop if decision == BLOCKED
stop if generation_review_round > 3
stop if debug_round > 2
```

状态回写字段

current_stage
generation_review_round
debug_round
latest_decision
human_gate_status
artifact_status

评审结论语义

PASS: 可进入下一阶段
REVISE / REVISE_GENERATION: 返回实现侧
ENTER_DEBUG: 进入失败归因
BLOCKED: 停机等人类判断

角色矩阵：谁负责什么，不负责什么

Orchestrator

负责状态机、工件路径、
预算与调度
不做架构结论
不代替审核做 PASS 判定

Planner

产出 file-level 方案 / 测试矩阵 / 风险
定义 must-not-do 边界
不直接改代码

Implementer

按计划最小实现
写构建 / 测试日志
被审核驳回后进入再实现

Review trio

Spec: 看目标闭环
Code: 看质量 / 边界
Upstream: 看 patch 粒度与叙事

Debug

Failure-Analyzer
Fix-Agent
Regression-Guard

技术判断：把“实现”和“判定通过”拆给不同 Agent，是这套系统避免自证正确的核心。

工件契约：让上下文留在文件，不留在聊天里

Discover 工件

discover/code-evidence.md
discover/history-evidence.md
state/gap_registry.yaml
state/issue_map.yaml

Plan 工件

plans/design.md
plans/test-matrix.md
plans/risk.md

Generate 工件

logs/build-round-N.log
logs/test-round-N.log
state/change-summary-round-N.md

Patch 工件

series-cover-letter.md
checkpatch.txt
get-maintainer.txt

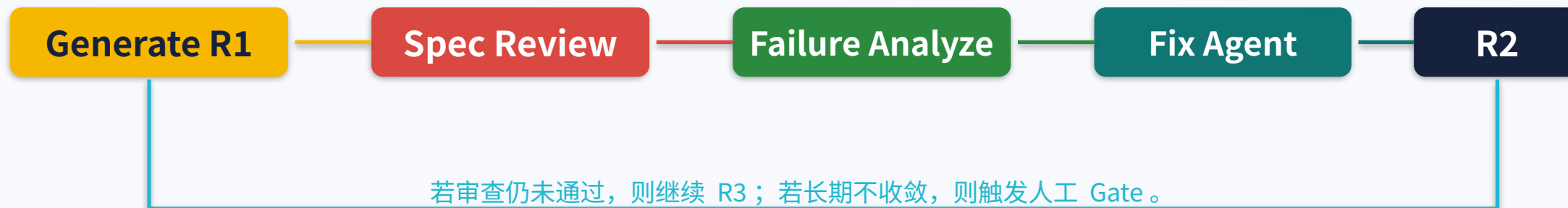
目录约定

```
artifacts/<issue>/  
  discover/  
  plans/  
  review/  
  debug/  
  patch/  
  state/  
  logs/
```

命名约定

```
run_history/<issue>-round-<n>.md  
review/spec-round-<n>.md  
debug/failure-analysis-round-<n>.md  
patches/<issue>/draft-round-<n>.patch
```

生成 / 审核 / 调试 闭环设计



为什么要单独 Debug

很多失败不是编译错，而是规格未闭环
Failure-Analyzer 更像根因分析器，不是修代码器

Round 记录必须包含

本轮目标
修改文件
测试结果
review decision
next step

停止条件

连续 3 轮不收敛
错误反复出现且根因不稳定
出现架构语义争议

质量闸门与验证矩阵

3

人工 Gate

2+

生成 / 审核轮次

1

issue per run

6+

关键工件

自动质量闸门

make ARCH=riscv defconfig
目标架构 build
kselftest relevant subset
kunit relevant subset
checkpatch.pl
get_maintainer.pl

人工 Gate 触发条件

问题定义不清
ABI/UAPI/DT/Kconfig 用户可见影响
maintainer rejection history
准备发信

评审视角

Spec: 需求是否闭环
Code: 边界与可维护性
Upstream: patch 叙事 / 粒度

成本、风险与工程控制

典型失败模式

把设计问题误判成代码问题
实现者过度保守只改一半
过度设计导致 patch 粒度膨胀
长日志回灌导致 token 成本失控

成本控制策略

每次只处理一个窄 issue
长上下文外置为工件路径
高歧义规划交给更强模型
实现闭环尽量在 cheaper
runtime 完成

收敛策略

round 上限
debug 上限
人工 Gate
单 issue 单分支

最终价值

把人工从重复劳动里解放出来
保留关键判断
形成可复盘工程资产

Demo Issue : riscv/kvm MMU Kconfig 一致

性修复

Issue

目标：让 dependency/help text 都显式表达 MMU 约束

范围：只触及 arch/riscv/kvm/Kconfig

风险：低，但属于 Kconfig 用户可见变化

Round 1

实现通过模拟构建检查
但 Spec Review 指出 help text 未闭环

结论：ENTER_DEBUG

Debug

Failure-Analyzer 认定为 spec gap

Fix-Agent 继续单文件修复
保持单 patch、可回滚

Round 2

补充 help text 的 MMU 说明
Spec Review Round 2: PASS
状态流转到 patch-ready

这个 demo 证明：审查失败并不一定意味着代码坏了，也可能只是“实现没有完整满足设计目标”。

技术评审结论

这套方案为什么可行

职责解耦：探索、规划、实现、审核、调试分层

状态显式：所有阶段都回写 workflow.yaml 与工件目录

闭环可控：round 上限 + 人工 Gate + patch-ready 定义

仍需谨慎的地方

架构语义争议

真实 lore 证据补齐

日志真实性与回归覆盖

发信礼仪与 maintainer 偏好

建议决策

先用 MVP 跑一个高置信度窄 issue

强制经历至少 2 轮生成 / 审核

把真实工具链日志接入现有工件目录

不是“多开模型”，而是“多角色互相制衡”